

WHAT WE LEARNED FROM ASSIGNMENT 5 TASKFLOW – PRODUCTIVITY DASHBOARD

1. INTRODUCTION

Assignment 5 was an important step from building simple Flutter interfaces towards developing a complete, professional, multi-screen application.

Through this assignment, we learned how to combine UI design, application logic, state management, navigation, responsiveness, animations, forms, timers, and reusable components into one practical Flutter project.

The project, TaskFlow, was designed as a productivity management application rather than a basic Todo application. This helped us understand how different Flutter concepts work together in a real-world application.

2. BUILDING A COMPLETE FLUTTER APPLICATION

One of the major things we learned was how to structure a complete Flutter application.

Instead of putting everything inside one file, the application was divided into different sections:

- Models
- Screens
- Widgets
- Utilities
- Application configuration

The project structure helped us understand the importance of separating application logic and UI components.

For example, the Task model is responsible for representing task data, while screens are responsible for displaying major application pages and widgets are responsible for reusable UI elements.

This makes the application easier to understand, maintain, and expand.

3. WORKING WITH STATEFULWIDGET

We learned how StatefulWidget can be used when the UI needs to change during application execution.

TaskFlow contains several dynamic values such as:

- Task completion status
- Number of active tasks
- Number of completed tasks
- Search results
- Selected filters
- Selected navigation page
- Focus timer state
- Theme state

These values can change based on user interaction.

We learned that calling `setState()` causes Flutter to rebuild the required UI so that the displayed information stays synchronized with the current application state.

This helped us understand the difference between static UI and dynamic UI.

4. SHARING DATA BETWEEN SCREENS

Another important concept learned was passing data and callbacks between different widgets and screens.

The main task collection is maintained by the main application screen and passed to screens that require it.

For example:

Dashboard → displays task statistics

Tasks → manages tasks

Calendar → displays task-related calendar information

Categories → displays task distribution

Callbacks are used for operations such as:

- Adding a task
- Completing a task
- Deleting a task
- Changing filters
- Changing search values

This taught us how different parts of an application can communicate without placing all the code in a single widget.

5. CREATING REUSABLE WIDGETS

We learned the importance of reusable components in Flutter.

Instead of repeatedly writing the same UI code, reusable widgets were created for commonly required elements.

Examples include:

- TaskCard
- FilterChips
- SearchBarWidget
- AddTaskSheet

For example, TaskCard can display different tasks while using the same layout and behavior.

Reusable widgets reduce duplicate code and make future modifications much easier.

6. RESPONSIVE UI DESIGN

A major learning outcome was responsive design.

A Flutter application should not assume that every user has the same screen size.

TaskFlow adapts its interface according to available screen width.

On desktop:

- Sidebar navigation is displayed.
- Content is arranged in multiple columns.
- Dashboard panels have more horizontal space.
- Larger productivity sections can be displayed together.

On mobile:

- A compact AppBar is used.
- Bottom NavigationBar is provided.
- Content is displayed vertically.
- Controls are adapted for smaller screens.

We learned how LayoutBuilder and screen constraints can be used to create different layouts for different devices.

7. APPLICATION NAVIGATION

We learned how to create navigation between different sections of an application.

TaskFlow contains:

- Dashboard
- Tasks
- Calendar
- Projects
- Categories
- Focus Timer

A selected page index is maintained by the application, and the displayed screen changes according to the selected navigation item.

We also learned how desktop and mobile applications can use different navigation patterns.

Desktop uses a sidebar, while mobile uses a bottom navigation bar.

8. FORMS AND USER INPUT

The Add Task functionality helped us learn how to collect and validate user input.

The task creation form contains:

- Task title
- Task description

Form validation was implemented to prevent invalid task titles.

We learned how to use:

- Form
- GlobalKey<FormState>
- TextFormField
- TextEditingController
- Validator functions

We also learned that controllers should be properly disposed when they are no longer required.

9. MODAL BOTTOM SHEETS

The Add Task functionality introduced the use of modal bottom sheets.

Instead of navigating to a completely separate page, the user can open a bottom sheet and create a task without leaving the current screen.

We learned how to use:

- showModalBottomSheet()
- isScrollControlled
- MediaQuery.viewInsets
- Form-based bottom sheet layouts

This provides a more modern and convenient user experience.

10. SEARCHING AND FILTERING

We learned how to dynamically filter a list based on user input.

The task search functionality checks the task title and description against the entered search query.

Filtering was also implemented using three options:

- All
- Active
- Completed

This taught us how to transform a list dynamically using conditions and collection operations.

The UI automatically updates when the search query or selected filter changes.

11. DELETE AND UNDO FUNCTIONALITY

The task deletion feature taught us how to provide safer user interactions.

Before deleting a task, a confirmation dialog is displayed.

After deletion, a SnackBar provides an UNDO action.

This introduced several useful Flutter concepts:

- AlertDialog
- SnackBar
- SnackBarAction
- Navigator.pop()
- List insertion and removal

We learned that good applications should not only perform actions but also protect users from accidental actions.

12. ANIMATIONS

Animations were used to make TaskFlow feel more interactive and polished.

We learned about:

- AnimationController
- AnimatedContainer
- AnimatedSwitcher
- AnimatedDefaultTextStyle
- ScaleTransition
- Circular progress animations

For example, completing a task changes its appearance with animation, while the completion icon transitions smoothly.

This demonstrated how animations can improve user experience without making the application unnecessarily complicated.

13. LIGHT AND DARK THEMES

We learned how to implement application-wide themes using ThemeData.

TaskFlow supports:

- Light Mode
- Dark Mode

Separate theme configurations were created for the two modes.

Colors, cards, input fields, buttons, borders, and other UI elements respond to the selected theme.

We learned that centralized theme configuration is much better than manually setting colors throughout every widget.

14. CUSTOM UI AND DATA VISUALIZATION

The Productivity Tracker introduced us to custom drawing in Flutter.

CustomPainter was used to draw a productivity graph.

We learned how Canvas and Paint can be used to draw:

- Lines
- Grid lines
- Points
- Curves

This showed us that Flutter is not limited to standard widgets and can also be used to create customized visual components.

15. WORKING WITH TIMERS

The Focus Timer helped us understand time-based application logic.

A Timer was used to update the countdown periodically.

The Focus Timer supports:

- Start
- Pause
- Reset
- Skip
- Focus mode
- Short Break mode
- Multiple duration presets

We learned how to start and cancel a Timer and how to update the UI when time changes.

We also learned the importance of disposing timers when a StatefulWidget is removed from the widget tree.

16. PROJECT ORGANIZATION AND CODE QUALITY

The assignment taught us that code organization becomes increasingly important as an application grows.

The project was separated into:

models/
screens/
widgets/
utils/

This separation makes it easier to locate specific functionality.

We also learned to avoid unnecessary duplicate widgets and to keep reusable UI components independent wherever possible.